

CHALLENGE #6

I process data using Python, which is read in data frame format to facilitate processing. Some libraries used are Pandas, Numpy, Seaborn, and Matplotlib. After the data is read, we first check this dataset's feature/column name because naming the feature/column is one of the most important things in Python before cleaning.

Many feature names still do not match the Python format, which recommends using feature names without spaces or punctuation marks other than (`_`). So, the first thing we do is change the feature name and delete some features that we don't need.

Changing the feature name format

We change the feature/column naming format using Python's rename method.

Before:

	Q1: GOING OUT?	Q2: GENDER	Q3: AGE	Q4: COUNTRY	Q5: STATE, PROVINCE, COUNTRY, ETC
0	NaN	NaN	NaN	NaN	NaN
1	No	Male	44	USA	NM
2	NaN	Male	49	USA	Virginia

After:

	going_out	sex	age	country	state
0	NaN	NaN	NaN	NaN	NaN
1	No	Male	44	USA	NM
2	NaN	Male	49	USA	Virginia

Then, we move to the next step, which is data cleaning.

Data Cleaning

We first check the NaN value using the code `df.isnull().sum()`.

It can be seen that this dataset is very messy and needs a lot of cleaning; the data has many missing values (NaN). Then we delete columns that have too many NaN values. The column that is deleted here is the column that has more than 80% NaN values from the total data.

	Column	NaN Count	Percentage NaN
113	Unnamed: 113	2451	99.634146
118	yahoo	2393	97.276423
115	dailydish	2375	96.544715
117	espn	2361	95.975610
111	Q9: OTHER COMMENTS	2074	84.308943
110	despair_other	1743	70.853659
109	joy_other	1549	62.967480
116	science	1098	44.634146
45	Q6 JoyJoy (Mit Iodine!)	1026	41.707317
58	Q6 Maynards	1024	41.626016

We can see that the column/feature to be deleted is Unnamed:113, Yahoo (Q12), Daily Dish (Q12), ESPN (Q12), and Q9:Other Comments.

- **Q1: Going Out.** At the column `going_out` there are 3 values including "NaN", "No", and "Yes". The value of NaN can be replace with "Maybe".
- **Q2:Gender.** Before. It can be seen that there are 5 unique values in gander, here it is chosen to replace the values 'I'd rather not say' and 'Other' with 'Didn't say'. Then the NaN value is also filled with 'Didn't say'.After
- **Q3: Age**

Before. It can be seen that there are non-numeric values where the age column should contain numeric values. So the first thing to do is to replace the non-numeric values with NaN values. After that, all NaN values will be replaced with the Median value from the age column.

```
df['age'].unique()
```

```
array([nan, '44', '49', '40', '23', '53', '33', '43', '56', '64', '37',  
       '59', '48', '54', '36', '45', '25', '34', '35', '38', '58', '50',  
       '47', '16', '52', '63', '65', '41', '27', '31', '61', '46', '42',  
       '62', '29', '39', '32', '28', '69', '67', '30', '22', '26', '51',  
       '70', '24', '18', '19', 'Old enough', '57', '60', '66', '12',  
       'Many', '55', '72', '?', '21', '11', 'no', '9', '68', '20', '6',  
       '10', '71', '90', '13', '45-55', '312', '99', '7', 'hahahahaha',  
       '88', '39.4', '74', '102', 'older than dirt', '17', '15', '8',  
       '75', '5u', 'Enough', 'See question 2', '24-50', '14', 'Over 50',  
       '100', '76', '1000', 'sixty-nine', '46 Halloweens.', 'ancient',  
       '77', 'OLD', 'old', '73', '70 1/2', '1', 'MY NAME JEFF', '4',  
       '59 on the day after Halloween', 'old enough', 'your mom',  
       'I can remember when Java was a cool new language', '60+'],  
      dtype=object)
```

After

```
df['age'].unique()
array([41, 44, 49, 40, 23, 53, 33, 43, 56, 64, 37, 59, 48, 54, 36, 45, 25,
       34, 35, 38, 58, 50, 47, 16, 52, 63, 65, 27, 31, 61, 46, 42, 62, 29,
       39, 32, 28, 69, 67, 30, 22, 26, 51, 70, 24, 18, 19, 57, 60, 66, 12,
       55, 72, 21, 11, 68, 20, 10, 71, 13, 74, 17, 15, 14, 73])
```

• Q4:Country

In the country column are 64 NaN values and several wrong country names or the name of a country with different names such as USA, America, U.S.A. To clean up the wrong naming, it is necessary to standardise the writing of country names in the dataset. First, we changed the entire country column value to lowercase letters. Using fuzzy matching technique from fuzzywuzzy library to correct variations in writing or typos. Of course, we need to list country names that we will refer to checking. Meanwhile, the NaN value will replace the word 'other'. Then, because we are asked to filter between America and not, we can replace countries other than America with 'not America'.

```
df['country'].unique()
```

```
array(['other', 'usa', 'canada', 'uk', 'mexico', 'france', 'netherlands',
       'germany', 'australia', 'ireland', 'japan', 'scotland',
       'switzerland', 'china'], dtype=object)
```

	count
country	
usa	2062
not america	398

dtype: int64

• Q5: State.

In the state column, the value is also checked with print(df['state'].nunique()), and the result is 511. Because the unique value in the state is too diverse and the state column is represented by country, it can be removed. The same process can be done on the country column.

• Q6 all columns.

Q6 is a column with types of candies and contains data on people's reviews of these candies. Each column contains four unique values: 'Meh', 'Despair', 'Joy', and NaN. The NaN value in all Q6 columns can be replaced with 'Unknown', which is considered an unknown reaction.

• Q7:Joy_Other and Q8:Despair_Other. The data contents of these two columns are long sentences.

```
df['joy_other'].unique()
array([nan, 'Mounds',
       "Reese's crispy crunchy bars, 5th avenue bars, ferrero rocher, dries fruit other than raisins",
       'using my elbows to get in front of the little kids in line.',
       'Peanut butter m&ms ', 'Quality brand dark chocolate',
       'Cheesy Snax', 'Cadbury Fingers', 'Sour keys',
       'Payday, baby Ruth ', 'Any full size candy bar', 'Beer',
       'Mounds, Almond Joy (it's in the name, for god's sake)',
       'Chunky, Marathon Bar, Caramello, Almond Joy, Mounds, Charleston Chew',
       'Puppies, pure sunshine, kombucha, and a kiss from my one true love',
       'Jelly Belly jelly beans!',
       'Cow Tails! Butterscotch! Sugar Babies!', 'Puppies',
       'Marathon Bars (in England they are called Curly Wurries). Pop rocks & Space dust, Almond Joy',
       'Almost anything from Lindt', 'SIXLETS, WHOPPERS, ZERO',
       'raw eggs and a map of neighborhood houses not giving out candy',
       'A cappuccino',
       'Any chocolate covered fruit, those orange-flavored chocolate orbs that you crack into slices, chocolate covered pretzels',
       'hot tamales or other cinnamon candies',
       'Exotic international candies',
       'Belly Flops (Irregular Jelly Belly from the Dollar Store)',
       'Brussels sprouts, canned spinach, canned SPAM', 'Tootsie Rolls',
       'Warheads', 'Almond Joy, Peanut Butter M&Ms, Bullseyes',
       'Family, granddaughter',
```

we use the Counter library from collections and then take the three most important words.

```
from collections import Counter
def find_important_words(text):
    if pd.isnull(text):
        return ""
    words = text.lower().split()
    word_counts = Counter(words)
    most_common_words = [word for word,
count in word_counts.most_common(3)]
    return ','.join(most_common_words)

df['important_joy_words'] =
df['joy_other'].apply(find_important_words)
```

```
df['important_joy_words']
important_joy_words
0
1 mounds
2
3 bars,,reese's,crispy
4
...
2455 mounds
2456
2457 tiny,bottles,of
2458
2459
```